Notes:

- Don't forget about HW0 if you haven't done it yet!
- HW1 due Oct 2.

Last class:

- More bash commands: `grep`, `ssh`, `scp`, `top`, `htop`, `ps`, `bg`, `fg`, `nohup`
- Customizing environment: `.bashrc`, `export`, `alias`
- VSCode
- Markdown
- Basic Python

Today:

- More basic Python
- Start `git`

Last class, we talked about basic arithmetic operations in Python. Continuing that, by default division is "true division"

In [1]:
```python
print("1/2 is ", 1/2)
```
```
1/2 is  0.5
```

In [2]:
```python
print("1//2 is ", 1//2)
```
```
1//2 is  0
```

In [3]:
```python
print("3//2 is ", 3//2)
```
```
3//2 is  1
```

In [4]:
```python
print("1%2 is ", 1%2)
```
```
1%2 is  1
```

Use `//` for integer division, and `%` for modulus (remainder)

Can do complex numbers, too

In [5]:
```python
c_num = 1 + 2.2*1j
print(c_num.real)
print(c_num.imag)
```
```
1.0
2.2
```

Can use `2j` or `3j`, etc. for integer multiples of $j = \sqrt{-1}$, but use `1j` as imaginary unit in multiplication

In a Jupyter notebook, can add comments in markdown cells, but can also add comments in Python

In [6]:
```python
# Just a comment
2+3 # End of line comment
```
Out[6]:
```
5
```

For longer comments, use triple quotes:

In [7]:
```python
'''
This is
a big
comment
'''
2+3
```
Out[7]:
```
5
```

Python functions are units of code that take input arguments, do something, and return a result.

We've already seen some functions: `print`, also `type`, `len`

In [8]:
```python
print("stuff")
```
```
stuff
```

Here, `"stuff"` is the argument. Can also call print with no arguments:

In [9]:
```python
print() # Still a function, so need parens
```

The output from a print statement is not its result:

In [10]:
```python
result = print("stuff")
print(result)
```
```
stuff
None
```

Python uses `None` as a special value where no numerical or other value makes sense

Some functions take more than 1 argument:

In [11]:
```python
max(2, 4, -3)
```
Out[11]:
```
4
```

And some functions take arguments of different types:

In [12]:
```python
max('a', 'B', '7')
```
Out[12]:
```
'a'
```

For letters, `max` sorts as 0-9 first , then A-Z, then a-z, for the order

But cannot mix types:

In [13]:
```python
max(3, '4')
```
```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[13], line 1
----> 1 max(3, '4')

TypeError: '>' not supported between instances of 'str' and 'int'
```

Some functions have default arguments:

In [14]:
```python
pi = 3.14159265
round(pi)
```
Out[14]:
```
3
```

In [15]:
```python
round(pi, 2)
```
Out[15]:
```
3.14
```

Some data types in Python have **methods**, functions that act on their data

In [16]:
```python
my_string = "Hello there"
print(my_string.swapcase())
```
```
hELLO THERE
```

Built-in functions have help data:

In [17]:
```python
help(round)
```
```
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.  Otherwise
    the return value has the same type as the number.  ndigits may be negative.
```

Python reports errors when we make mistakes

In [18]:
```python
age = = 25
```
```
  Cell In[18], line 1
    age = = 25
          ^
SyntaxError: invalid syntax
```

Note that Python has good error reporting, with different messages for `SyntaxError`, `TypeError`, `NameError`, etc.

- useful when writing and debugging functions

Much of the power of Python comes from the many libraries that provide functions and data for us to use.

In Python, libraries are collections of modules, and functions/data are contained in modules.

There is very extensive library that is always available in Python, called the Python Standard Library.

A few of its modules are:

- `datetime` module, for basic date and time functions (timing code)
- `math` module, for mathematical functions
- `os` module, for operating system interfaces
- `argparse` module, to handle command-line arguments
- `unittest` module, for testing code correctness
- `pdb` module, the python debugger

We'll dive into lots of these (and other libraries) later in the course. For now, just use `math` module as an example

In order to use a module in Python, we must first import it.

In [19]:
```python
import math
print('pi is ', math.pi)
```
```
pi is  3.141592653589793
```

Use `module_name.thing_name` to refer to functions or data from a module, unless we load it differently.

In [20]:
```python
import math as m
print("pi is ", m.pi)
```
```
pi is  3.141592653589793
```

Can give modules "shorthand" names with `import module as shorthand`

Can also import specific items from a module with no prefix:

In [21]:
```python
from math import pi
print("pi is ", pi)
```
```
pi is  3.141592653589793
```

Can also just import everything from a module with no prefix:

In [22]:
```python
from math import *
print("cos(pi) is ", cos(pi))
```
```
cos(pi) is  -1.0
```

These star imports are convenient, but dangerous when you're using many modules. Nothing stops 2 modules from having different functions with same name. Which one do you get? (Import order matters!)

Most modules have good documentation

In [23]:
```python
help(math)
```
```
Help on module math:

NAME
    math

DESCRIPTION
    This module provides access to the mathematical functions
    defined by the C standard.

FUNCTIONS
    acos(x, /)
        Return the arc cosine (measured in radians) of x.

        The result is between 0 and pi.

    acosh(x, /)
        Return the inverse hyperbolic cosine of x.

    asin(x, /)
        Return the arc sine (measured in radians) of x.

        The result is between -pi/2 and pi/2.

    asinh(x, /)
        Return the inverse hyperbolic sine of x.

    atan(x, /)
        Return the arc tangent (measured in radians) of x.

        The result is between -pi/2 and pi/2.

    atan2(y, x, /)
        Return the arc tangent (measured in radians) of y/x.

        Unlike atan(y/x), the signs of both x and y are considered.

    atanh(x, /)
        Return the inverse hyperbolic tangent of x.

    cbrt(x, /)
        Return the cube root of x.

    ceil(x, /)
        Return the ceiling of x as an Integral.

        This is the smallest integer >= x.

    comb(n, k, /)
        Number of ways to choose k items from n items without repetition and without order.

        Evaluates to n! / (k! * (n - k)!) when k <= n and evaluates
        to zero when k > n.

        Also called the binomial coefficient because it is equivalent
        to the coefficient of k-th term in polynomial expansion of the
        expression (1 + x)**n.

        Raises TypeError if either of the arguments are not integers.
        Raises ValueError if either of the arguments are negative.

    copysign(x, y, /)
        Return a float with the magnitude (absolute value) of x but the sign of y.

        On platforms that support signed zeros, copysign(1.0, -0.0)
        returns -1.0.

    cos(x, /)
        Return the cosine of x (measured in radians).

    cosh(x, /)
        Return the hyperbolic cosine of x.

    degrees(x, /)
        Convert angle x from radians to degrees.

    dist(p, q, /)
        Return the Euclidean distance between two points p and q.

        The points should be specified as sequences (or iterables) of
        coordinates.  Both inputs must have the same dimension.

        Roughly equivalent to:
            sqrt(sum((px - qx) ** 2.0 for px, qx in zip(p, q)))

    erf(x, /)
        Error function at x.

    erfc(x, /)
        Complementary error function at x.

    exp(x, /)
        Return e raised to the power of x.

    exp2(x, /)
        Return 2 raised to the power of x.

    expm1(x, /)
        Return exp(x)-1.

        This function avoids the loss of precision involved in the direct evaluation of exp(x)-1 for small x.

    fabs(x, /)
        Return the absolute value of the float x.

    factorial(n, /)
        Find n!.

        Raise a ValueError if x is negative or non-integral.

    floor(x, /)
        Return the floor of x as an Integral.

        This is the largest integer <= x.

    fma(x, y, z, /)
        Fused multiply-add operation.

        Compute (x * y) + z with a single round.

    fmod(x, y, /)
        Return fmod(x, y), according to platform C.

        x % y may differ.

    frexp(x, /)
        Return the mantissa and exponent of x, as pair (m, e).

        m is a float and e is an int, such that x = m * 2.**e.
        If x is 0, m and e are both 0.  Else 0.5 <= abs(m) < 1.0.

    fsum(seq, /)
        Return an accurate floating-point sum of values in the iterable seq.

        Assumes IEEE-754 floating-point arithmetic.

    gamma(x, /)
        Gamma function at x.

    gcd(*integers)
        Greatest Common Divisor.

    hypot(...)
        hypot(*coordinates) -> value

        Multidimensional Euclidean distance from the origin to a point.

        Roughly equivalent to:
            sqrt(sum(x**2 for x in coordinates))

        For a two dimensional point (x, y), gives the hypotenuse
        using the Pythagorean theorem:  sqrt(x*x + y*y).

        For example, the hypotenuse of a 3/4/5 right triangle is:

            >>> hypot(3.0, 4.0)
            5.0

    isclose(a, b, *, rel_tol=1e-09, abs_tol=0.0)
        Determine whether two floating-point numbers are close in value.

          rel_tol
            maximum difference for being considered "close", relative to the
            magnitude of the input values
          abs_tol
            maximum difference for being considered "close", regardless of the
            magnitude of the input values

        Return True if a is close in value to b, and False otherwise.

        For the values to be considered close, the difference between them
        must be smaller than at least one of the tolerances.

        -inf, inf and NaN behave similarly to the IEEE 754 Standard.  That
        is, NaN is not close to anything, even itself.  inf and -inf are
        only close to themselves.

    isfinite(x, /)
        Return True if x is neither an infinity nor a NaN, and False otherwise.

    isinf(x, /)
        Return True if x is a positive or negative infinity, and False otherwise.

    isnan(x, /)
        Return True if x is a NaN (not a number), and False otherwise.

    isqrt(n, /)
        Return the integer part of the square root of the input.

    lcm(*integers)
        Least Common Multiple.

    ldexp(x, i, /)
        Return x * (2**i).

        This is essentially the inverse of frexp().

    lgamma(x, /)
        Natural logarithm of absolute value of Gamma function at x.

    log(...)
        log(x, [base=math.e])
        Return the logarithm of x to the given base.

        If the base is not specified, returns the natural logarithm (base e) of x.

    log10(x, /)
        Return the base 10 logarithm of x.

    log1p(x, /)
        Return the natural logarithm of 1+x (base e).

        The result is computed in a way which is accurate for x near zero.

    log2(x, /)
        Return the base 2 logarithm of x.

    modf(x, /)
        Return the fractional and integer parts of x.

        Both results carry the sign of x and are floats.

    nextafter(x, y, /, *, steps=None)
        Return the floating-point value the given number of steps after x towards y.

        If steps is not specified or is None, it defaults to 1.

        Raises a TypeError, if x or y is not a double, or if steps is not an integer.
        Raises ValueError if steps is negative.

    perm(n, k=None, /)
        Number of ways to choose k items from n items without repetition and with order.

        Evaluates to n! / (n - k)! when k <= n and evaluates
        to zero when k > n.

        If k is not specified or is None, then k defaults to n
        and the function returns n!.

        Raises TypeError if either of the arguments are not integers.
        Raises ValueError if either of the arguments are negative.

    pow(x, y, /)
        Return x**y (x to the power of y).

    prod(iterable, /, *, start=1)
        Calculate the product of all the elements in the input iterable.

        The default start value for the product is 1.

        When the iterable is empty, return the start value.  This function is
        intended specifically for use with numeric values and may reject
        non-numeric types.

    radians(x, /)
        Convert angle x from degrees to radians.

    remainder(x, y, /)
        Difference between x and the closest integer multiple of y.

        Return x - n*y where n*y is the closest integer multiple of y.
        In the case where x is exactly halfway between two multiples of
        y, the nearest even value of n is used. The result is always exact.

    sin(x, /)
        Return the sine of x (measured in radians).

    sinh(x, /)
        Return the hyperbolic sine of x.

    sqrt(x, /)
        Return the square root of x.

    sumprod(p, q, /)
        Return the sum of products of values from two iterables p and q.

        Roughly equivalent to:

            sum(itertools.starmap(operator.mul, zip(p, q, strict=True)))

        For float and mixed int/float inputs, the intermediate products
        and sums are computed with extended precision.

    tan(x, /)
        Return the tangent of x (measured in radians).

    tanh(x, /)
        Return the hyperbolic tangent of x.

    trunc(x, /)
        Truncates the Real x to the nearest Integral toward 0.

        Uses the __trunc__ magic method.

    ulp(x, /)
        Return the value of the least significant bit of the float x.

DATA
    e = 2.718281828459045
    inf = inf
    nan = nan
    pi = 3.141592653589793
    tau = 6.283185307179586

FILE
    /opt/homebrew/Cellar/python@3.13/3.13.3_1/Frameworks/Python.framework/Versions/3.13/lib/python3.13/lib-dynload/math.cpython-313-darwin.so
```

In [24]:
```python
help(math.cos)
```
```
Help on built-in function cos in module math:

cos(x, /)
    Return the cosine of x (measured in radians).
```

In [25]:
```python
# math.cos()  # VSCode pops up hints as you type commands
```

Can do lots of things in Python with standard library modules:

In [ ]:
```python
power = 5
print(math.pow(math.pi, power))
```
```
31.006276680299816
```

In a Jupyter notebook, can edit the cell to re-evaluate with different powers

When we do something many times, helpful to be able to do it in different ways.

We can write our own functions

In [29]:
```python
def pi_power(power = 2):
    print(math.pow(math.pi, power))
    return None
```

In [30]:
```python
pi_power(2)
```
```
9.869604401089358
```

In [31]:
```python
pi_power(1)
```
```
3.141592653589793
```

In [32]:
```python
pi_power(3)
```
```
31.006276680299816
```

In [33]:
```python
pi_power()
```
```
9.869604401089358
```

When we wrote function, specified default `power = 2`, so if we call with no arguments, we get output as if we called it with `2` as the argument

In [34]:
```python
pi_power
```
Out[34]:
```
<function __main__.pi_power(power=2)>
```

If we call a function without parentheses, Python assumes we want the function as an object, and not to evaluate the function

We can also write a function as a python script, to be run from the command line. To do this, we might want to take arguments at the command line, so we use `argparse`

Three pieces to using `argparse`:

1. Declare an instance of `argparse.ArgumentParser()`
2. List arguments, data types, and their default values
3. Call `parser.parse_args()` to read the values provided at the command line. These become data for the `args` object it returns

Can also add help info to both the parser itself, and to each argument

In [35]:
```python
import argparse
help(argparse.ArgumentParser)
```
```
Help on class ArgumentParser in module argparse:

class ArgumentParser(_AttributeHolder, _ActionsContainer)
 |  ArgumentParser(
 |      prog=None,
 |      usage=None,
 |      description=None,
 |      epilog=None,
 |      parents=[],
 |      formatter_class=<class 'argparse.HelpFormatter'>,
 |      prefix_chars='-',
 |      fromfile_prefix_chars=None,
 |      argument_default=None,
 |      conflict_handler='error',
 |      add_help=True,
 |      allow_abbrev=True,
 |      exit_on_error=True
 |  )
 |
 |  Object for parsing command line strings into Python objects.
 |
 |  Keyword Arguments:
 |      - prog -- The name of the program (default:
 |          ``os.path.basename(sys.argv[0])``)
 |      - usage -- A usage message (default: auto-generated from arguments)
 |      - description -- A description of what the program does
 |      - epilog -- Text following the argument descriptions
 |      - parents -- Parsers whose arguments should be copied into this one
 |      - formatter_class -- HelpFormatter class for printing help messages
 |      - prefix_chars -- Characters that prefix optional arguments
 |      - fromfile_prefix_chars -- Characters that prefix files containing
 |          additional arguments
 |      - argument_default -- The default value for all arguments
 |      - conflict_handler -- String indicating how to handle conflicts
 |      - add_help -- Add a -h/-help option
 |      - allow_abbrev -- Allow long options to be abbreviated unambiguously
 |      - exit_on_error -- Determines whether or not ArgumentParser exits with
 |          error info when an error occurs
 |
 |  Method resolution order:
 |      ArgumentParser
 |      _AttributeHolder
 |      _ActionsContainer
 |      builtins.object
 |
 |  Methods defined here:
 |
 |  __init__(
 |      self,
 |      prog=None,
 |      usage=None,
 |      description=None,
 |      epilog=None,
 |      parents=[],
 |      formatter_class=<class 'argparse.HelpFormatter'>,
 |      prefix_chars='-',
 |      fromfile_prefix_chars=None,
 |      argument_default=None,
 |      conflict_handler='error',
 |      add_help=True,
 |      allow_abbrev=True,
 |      exit_on_error=True
 |  )
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  add_subparsers(self, **kwargs)
 |      # ==================================
 |      # Optional/Positional adding methods
 |      # ==================================
 |
 |  convert_arg_line_to_args(self, arg_line)
 |
 |  error(self, message)
 |      error(message: string)
 |
 |      Prints a usage message incorporating the message to stderr and
 |      exits.
 |
 |      If you override this in a subclass, it should not return -- it
 |      should either exit or raise an exception.
 |
 |  exit(self, status=0, message=None)
 |      # ===============
 |      # Exiting methods
 |      # ===============
 |
 |  format_help(self)
 |
 |  format_usage(self)
 |      # =======================
 |      # Help-formatting methods
 |      # =======================
 |
 |  parse_args(self, args=None, namespace=None)
 |      # =====================================
 |      # Command line argument parsing methods
 |      # =====================================
 |
 |  parse_intermixed_args(self, args=None, namespace=None)
 |
 |  parse_known_args(self, args=None, namespace=None)
 |
 |  parse_known_intermixed_args(self, args=None, namespace=None)
 |
 |  print_help(self, file=None)
 |
 |  print_usage(self, file=None)
 |      # =====================
 |      # Help-printing methods
 |      # =====================
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from _AttributeHolder:
 |
 |  __repr__(self)
 |      Return repr(self).
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors inherited from _AttributeHolder:
 |
 |  __dict__
 |      dictionary for instance variables
 |
 |  __weakref__
 |      list of weak references to the object
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from _ActionsContainer:
 |
 |  add_argument(self, *args, **kwargs)
 |      add_argument(dest, ..., name=value, ...)
 |      add_argument(option_string, option_string, ..., name=value, ...)
 |
 |  add_argument_group(self, *args, **kwargs)
 |
 |  add_mutually_exclusive_group(self, **kwargs)
 |
 |  get_default(self, dest)
 |
 |  register(self, registry_name, value, object)
 |      # ====================
 |      # Registration methods
 |      # ====================
 |
 |  set_defaults(self, **kwargs)
 |      # ==================================
 |      # Namespace default accessor methods
 |      # ==================================
```

In [36]:
```python
help(argparse.ArgumentParser().add_argument)
```
```
Help on method add_argument in module argparse:

add_argument(*args, **kwargs) method of argparse.ArgumentParser instance
    add_argument(dest, ..., name=value, ...)
    add_argument(option_string, option_string, ..., name=value, ...)
```

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python3 pi_power.py
9.869604401089358
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python3 pi_power.py --power 3
31.006276680299816
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python3 pi_power.py
9.869604401089358
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python3 pi_power.py --help
usage: pi_power.py [-h] [--power POWER]

Computes powers of pi

options:
  -h, --help     show this help message and exit
  --power POWER
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python3 pi_power.py --help
usage: pi_power.py [-h] [--power POWER]

Computes powers of pi

options:
  -h, --help     show this help message and exit
  --power POWER  Integer power of pi to compute
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python3 pi_power.py --power 3
```

Writing a program as a script lets us run it from the command line, so taking these command-line arguments is very useful! Also lets us submit Python codes to supercomputer job schedulers in a way where we can reproduce output very easily

We rarely write standalone python scripts that we run once and walk away from.

When we want to:

- share codes
- back up codes
- run codes on different machines
- have a record of changes as we make them

It is helpful to have code in some version control environment.

What's version control?

- when we work on software projects (large or small), need some way to record past history of changes in the code

To some extent, you can do this manually. When a "version" is ready, make an archival copy, then store that separately from code that is actively developed.

Version control software aims to automate this process and provide substantially more functionality to support collaboration.

Original systems were very basic:

- have central "repository" that stores a file and information on its past edits.
- a user "checks out" the file, edits it, then "checks in" the modified version.
- RCS (revision control system) worked only with single files, only single user could check out the file at a time.
    - no concurrency

Modern systems are very functional, with collaborative tools at their core.

Commonly used tools are subversion (svn), Mercurial, Microsoft Azure, and git

git has over 85% market share.

- combination of design/functionality (e.g., much faster for some operations than Mercurial) & popularity of cloud-based source code hosting sites like GitHub (owned by Microsoft), GitLab, and Bitbucket (owned by Atlassian)

At its core, git is a set of command-line tools that we interact with through the Bash shell (although graphical user interfaces also exist)

git commands take form `git action options`, where

- `action` is what you want to do, and
- `options` give additional information as needed.

```
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% git config --list
credential.helper=osxkeychain
init.defaultbranch=main
core.editor=emacs -nw
user.name=Scott MacLachlan
user.email=smaclachlan@mun.ca
push.default=simple
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
pull.rebase=false
```

To change settings globally, use `git config --global`

```
math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% git config --global user.name "Fred Flintstone"
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% git config --list
credential.helper=osxkeychain
init.defaultbranch=main
core.editor=emacs -nw
user.name=Fred Flintstone
user.email=smaclachlan@mun.ca
push.default=simple
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
pull.rebase=false
```

Common to set `user.name` and `user.email` when you interact across systems.

When you interact across platforms (especially with windows and anything else), common frustration with how windows and unix-based systems encode newlines in the text. So you might set `core.autocrlf` to resolve this conflict.

```
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% git config --global core.autocrlf true
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% git config --list
credential.helper=osxkeychain
init.defaultbranch=main
core.editor=emacs -nw
core.autocrlf=true
user.name=Fred Flintstone
user.email=smaclachlan@mun.ca
push.default=simple
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
pull.rebase=false
```

Important to set an editor that you know how to use. `vim` is often the default editor for git commands. Use `core.editor` for this

```
math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% git config --global core.editor "nano -w"
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% git config --list
credential.helper=osxkeychain
init.defaultbranch=main
core.editor=nano -w
core.autocrlf=true
user.name=Fred Flintstone
user.email=smaclachlan@mun.ca
push.default=simple
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
pull.rebase=false
```

When we create a respository, need a default name for "main branch", so can set this as well, with `init.defaultbranch`. Used to be common to use `master`, newer projects tend to use `main`.

Complete (but often cryptic) help via `git help` or `git config --help`